

A Mean-Field Game of Limit Order Book Dynamics

Filippo Antonio Cadorin

Advisor: Michele Fabi

May 2026

Abstract. The shape of a limit order book (LOB), depth as a function of price, is typically taken as exogenous in microstructure models. We develop a framework based on mean-field Bertrand competition and trader valuation heterogeneity, the framework is for a general asset; the bounded price contract is one clean illustration. The model produces a full LOB shape derived from strategic primitives without imposing any exogenous functional form on depth. It demonstrates that belief heterogeneity and Liquidity Provider (LP) patience are sufficient to construct a book from scratch; serving as a basis for a general LOB model.

Contents

1	Introduction	3
1.1	Motivation	3
1.2	Literature Review	3
1.3	Outline	4
2	Mean-Field Games in General	4
2.1	From symmetric Nash equilibria to the mean-field limit	5
2.2	Why this is useful for economic modelling	5
2.3	The MFG equilibrium structure	6
2.4	Connection with the limit order book application	8
2.5	Itô–Lions Lemma with Quantile Dependence	9
3	A Model of Limit-Order-Book Dynamics	10
3.1	The Asset	10
3.2	Transaction Cost	10
3.3	Trader Population	11
3.4	Trader Classification	11
3.5	Motivation for a Unified Population	11
3.6	Input Parameters	12
4	Applied Mean-Field Game	12
4.1	Trader Value Function and HJB Equation	13
4.2	Kolmogorov Forward Equation and Mean-Field Nash Equilibrium	14
4.3	Equilibrium Conditions	14
5	Algorithm	15
5.1	Problem Setting	15
5.2	Architecture	16
5.3	Rank Grid	17
5.4	Inner Loop: Policy Iteration	17
5.5	Middle Loop: Quantile Iteration	18
5.6	Outer Loop: Bertrand Fixed Point	19
6	Results	20
6.1	Densities and Book Shape	21
7	Conclusion	23
8	Appendix	24
8.1	Algorithm Code	24

1. Introduction

This section motivates the problem of deriving limit order book shape from fundamentals, situates our contribution within the existing literature on mean-field games and market microstructure, and outlines the structure of the report.

1.1. Motivation

LOBs are a central data structure in modern markets, they represent standing inventory of buy and sell limit orders at each price. Their shape, defined as depth as a function of price, is the object of interest in this report. Most existing models treat the LOB shape as exogenous or fitted to data. The question of how market fundamentals determine this shape is largely unanswered.

The following question arises: can we deduce the shape of a LOB from fundamentals, such as heterogeneous liquidity provider (LP) patience and valuation distribution, rather than assuming it? If so, what kind of LOB shape emerges as LPs engage in Bertrand competition for order flow?

We aim to adapt the [Ma and Noh \(2020\)](#) game framework, which derives LOB equilibrium from a Bertrand game among LPs, and specialize it to a theoretical asset setting.

At the end of this report we present a model that, despite limitations, shows that it is possible to construct from scratch the LOB shape from fundamentals, without taking any LOB as input.

1.2. Literature Review

Strand 1: LOB microstructure and reduced form models. The dominant approach to modelling LOBs treats order arrivals as exogenous stochastic processes. [Cont et al. \(2010\)](#), treat the book as a continuous time Markov chain with Poisson order flow, estimable from data. [Avellaneda and Stoikov \(2008\)](#) and [Guéant et al. \(2013\)](#), take the LOB dynamics as given and solve the market maker's inventory control problem against it. These models are powerful for execution and inventory management, but they take the shape of the book as an exogenous input, rather than deriving it from the behaviour of the agents who post liquidity. They cannot explain why the book has the shape it does.

Strand 2: Bertrand/Cournot mean-field games in finance. Our approach is complementary to those of Strand 1, we aim to derive market outcomes such as the LOB shape from strategic competition rather than assuming exogenous dynamics. Mean-field games ([Lasry and Lions, 2007](#); [Huang et al., 2006](#)) make this tractable in the large population limit. [Chan and Sircar \(2015\)](#) develop continuous time Bertrand and Cournot competition as mean-field games, with equilibrium characterized by a coupled forward–backward PDE system (KFE + HJB). [Ma and Noh \(2020\)](#) specialize this mean-field game approach to the LOB. The central insight builds on [Ma et al. \(2015\)](#): the best ask is identified with the value function of a representative seller's mean-field control problem, arising as the limit of a Bertrand game among liquidity providers. They formulate this as a reflected mean-field SDE with state-dependent jump intensities and show the value function is a viscosity solution of the HJB, this derives the equilibrium object that determines the LOB shape from primitives, rather than assuming it.

Strand 3: Limits to arbitrage and information-theoretic bounds. A natural question is why liquidity providers don't compete prices all the way to the no-arbitrage bound, or why mispricings

persist. As in [Shleifer and Vishny \(1997\)](#) real arbitrage requires capital and is risky, conducted by specialized agents with finite resources, so mispricings can persist. [Gromb and Vayanos \(2010\)](#), survey how funding and collateral constraints bound the liquidity arbitrageurs supply. Per [Touzo et al. \(2021\)](#) in the [Glosten and Milgrom \(1985\)](#) model, an analogy with the Szilárd engine yields a bound on informed-trader gains, $\mathbb{E}[G] \leq T \cdot H[Y]$, a no free lunch ceiling tied to information content and market noise. These literatures establish that liquidity provision is bounded and why; the equilibrium LOB shape we derive can be read as a quantitative description of the book that emerges once such frictions (LP impatience, heterogeneous beliefs, platform fees) are in force.

We build on this Ma-Noh framework, considering a single belief distribution simultaneously governs market order flow, LP cost bases, and the buyer/seller boundary. heterogeneous LP discount rates and buyer valuations enter as primitives. The equilibrium LOB shape is computed via a quantile fixed point algorithm, without imposing any exogenous functional form on depth, demonstrating that a full book can be constructed from strategic primitives.

1.3. Outline

The remainder of the report is organized as follows. The following section introduces the mean-field game framework and the rank coupling simplification. Next, we derive the Hamilton-Jacobi-Bellman equation following the Ma-Noh framework. Following that, we present the quantile fixed point algorithm used to compute the LOB equilibrium. Finally, we treat an abstract example and report illustrative numerical results, concluding with a discussion of possible extensions.

2. Mean-Field Games in General

Mean-field games provide a way of studying strategic interaction in environments with many similar agents. The starting point is standard non-cooperative game theory. In an N -player game, each agent chooses a strategy taking as given the strategies of the other $N - 1$ agents, and a Nash equilibrium is a profile in which no individual agent wants to deviate. This formulation is conceptually natural, but it becomes difficult to use when the number of players is large. A trader in a limit order book, for example, does not usually need to track the identity and full strategy of every other trader. What matters is the aggregate state of the market: how much liquidity is posted at different prices, how traders are distributed across inventory positions, and how this distribution evolves over time.

Mean-field game theory formalizes this idea by replacing the vector of opponents' actions with a population distribution. Instead of solving an N -dimensional strategic problem, one studies a representative agent who optimizes against a measure μ_t describing the cross-sectional distribution of states in the population. The agent is individually negligible: she takes μ_t as given when solving her control problem. At the same time, the distribution is endogenous, because it is generated by the behaviour of all agents when they use the equilibrium policy.

2.1. From symmetric Nash equilibria to the mean-field limit

Consider a symmetric dynamic game with N agents. Agent i has state $X_t^{i,N}$ and chooses a control $a_t^{i,N}$. The empirical distribution of states is

$$\mu_t^N = \frac{1}{N} \sum_{j=1}^N \delta_{X_t^{j,N}}.$$

In a symmetric Nash equilibrium, each player solves a best-response problem taking the other players' strategies as given. When agents are exchangeable and interact through aggregates, the payoff and transition law of player i can often be written as functions of her own state, her own action, and the empirical distribution μ_t^N . Thus the relevant strategic object is not the full vector $(X_t^{1,N}, \dots, X_t^{N,N})$, but the distribution of the population.

The mean-field approach studies the limit of this problem as $N \rightarrow \infty$. Under appropriate regularity conditions, the empirical distribution μ_t^N converges to a deterministic or conditionally deterministic measure flow μ_t . A single agent then has a negligible effect on the aggregate distribution, so the agent treats μ_t parametrically when choosing her control. The limiting control problem takes the schematic form

$$\sup_a \mathbb{E} \left[\int_0^T r(X_t, a_t, \mu_t) dt + G(X_T, \mu_T) \right]$$

subject to a state equation whose coefficients may also depend on μ_t . The representative agent's value function can be written as $V(t, x; \mu)$, where x is the agent's own state and μ summarizes the strategic environment created by the population.

The equilibrium condition closes the model. If the representative agent optimizes against a candidate flow $(\mu_t)_{t \geq 0}$ and uses the resulting optimal policy $a^*(t, x, \mu_t)$, the law of the state generated by that policy must reproduce the same flow μ_t . This fixed point is the mean-field analogue of Nash equilibrium. It can be interpreted as the limit of symmetric Nash equilibria in large anonymous games: no individual agent internalizes her own effect on the aggregate, but the aggregate is nevertheless determined by the collection of individual decisions.

2.2. Why this is useful for economic modelling

For economists, the appeal of MFGs is that they preserve strategic behaviour while making large-population models tractable. The framework is useful when the object of interest is not only an average action or a representative price, but the entire distribution of states. In market applications, this is essential. Liquidity, inventory, valuations, and order placement are all cross-sectional objects. The shape of a limit order book is itself a distributional outcome: it records how much liquidity is supplied across prices. A model that only tracks a representative trader cannot explain this shape, while a finite game with thousands of strategic traders is difficult to solve directly.

The MFG formulation also makes distributional comparative statics possible. Since equilibrium is described by a law μ_t , one can ask how changes in fundamentals shift the entire cross-section rather than only a mean. For example, in a market microstructure setting one can study whether a change in patience, execution risk, or trader valuations thickens the book near the best ask, spreads liquidity over a wider price range, or generates concentration at particular ranks of the distribution. These

questions are naturally distributional and are hard to formulate in a standard representative-agent model.

A further advantage is computational. Once the finite-player interaction is replaced by a representative-agent problem plus a consistency condition, the equilibrium can often be approximated by iterating between two steps. First, fix a candidate distribution and solve the agent's dynamic optimization problem. Second, propagate the population distribution implied by the resulting policy. Repeating these steps until convergence gives a constructive numerical procedure. This is especially attractive in applications, because the equilibrium object is not just a closed-form characterization but a simulable system.

2.3. The MFG equilibrium structure

The standard MFG system has two components. The first is a Hamilton–Jacobi–Bellman equation describing the representative agent's optimal control problem given the population distribution. In schematic form,

$$0 = \partial_t V(t, x) + \sup_a \{r(x, a, \mu_t) + \mathcal{L}^{a, \mu_t} V(t, x)\}, \quad (2.1)$$

where $\mathcal{L}^{a, \mu_t}$ is the generator of the individual state process when the agent chooses action a and the population state is μ_t . The second component is a Kolmogorov forward equation describing how the distribution evolves when all agents use the optimal policy:

$$\partial_t \mu_t = \left(\mathcal{L}^{a^*, \mu_t} \right)^* \mu_t, \quad (2.2)$$

where $(\mathcal{L}^{a^*, \mu_t})^*$ is the adjoint operator. Equation (2.1) gives individual optimality; equation (2.2) gives aggregate consistency. Together, they replace the system of N coupled best-response problems in the original game.

This representation is closest to the standard dynamic-programming logic familiar from stochastic control. Holding the population path $(\mu_t)_{t \geq 0}$ fixed, the individual agent solves an ordinary control problem in the private state x . The resulting optimal feedback $a^*(t, x, \mu_t)$ is then inserted into the forward equation, which verifies that the distribution generated by the optimizing agents is indeed the distribution that each agent took as given. The equilibrium is therefore a fixed point between individual optimality and aggregate law consistency.

A more general formulation makes explicit that the value function itself may depend on the distribution argument. The relevant object is then not simply $V(t, x)$, but a functional $V(t, x, \mu)$, where μ summarizes the cross-sectional state of the population. Applying Itô's formula to such an object requires more than the usual derivatives with respect to time and the individual state. The value changes both because the agent's own state moves, $X_t \mapsto X_{t+dt}$, and because the population distribution moves, $\mu_t \mapsto \mu_{t+dt}$. The first channel is captured by the standard derivatives $\partial_x V$ and $\partial_{xx} V$. The second channel requires a derivative with respect to the measure argument.

The appropriate derivative is the *Lions derivative*. Heuristically, $\partial_\mu V(t, x, \mu)(y)$ measures the sensitivity of the value function at (t, x, μ) to a marginal change in the population distribution around the point y . The additional argument y is important: it represents a possible state of another agent in the population. Thus, while $\partial_x V$ measures the effect of moving the representative agent's own state,

$\partial_\mu V$ measures the effect of changing the cross-sectional distribution of other agents' states.

One way to see how this derivative is computed is to perturb the distribution. Fix two probability measures μ and μ' , where μ' represents a nearby population distribution obtained, for example, by shifting a small amount of mass from one region of the state space to another. Connect the two distributions by the linear path

$$\mu^p := (1 - p)\mu + p\mu', \quad p \in [0, 1].$$

The change in the value functional can then be written, at first order, by integrating the derivative along this path:

$$V(t, x, \mu') - V(t, x, \mu) \approx \int_0^1 \int \partial_\mu V(t, x, \mu^p)(y) (\mu' - \mu)(dy) dp.$$

This formula gives the economic interpretation of the measure derivative. The term $(\mu' - \mu)(dy)$ describes how the population distribution is being perturbed at state y , while $\partial_\mu V(t, x, \mu^p)(y)$ gives the marginal effect of that perturbation on the representative agent's value. The integral over y aggregates these marginal effects across all possible states in the population. The integral over $p \in [0, 1]$ averages the derivative along the path from the initial distribution μ to the perturbed distribution μ' . In this sense, the Lions derivative plays the same role for distributions that an ordinary gradient plays for finite-dimensional state variables.

This also explains why independent copies appear in the notation. Since the measure μ_t is the law of the representative state X_t , integrals with respect to μ_t can be represented as expectations over a random variable with that same law. Formally, let $\tilde{X}_t$ be an independent copy of X_t , meaning that

$$\tilde{X}_t \sim \mu_t \quad \text{and} \quad \tilde{X}_t \text{ is independent of } X_t.$$

Then, for any suitable function g ,

$$\int g(y) \mu_t(dy) = \mathbb{E}[g(\tilde{X}_t)].$$

The tilde notation is therefore only a device for writing population averages. The variable $\tilde{X}_t$ should not be interpreted as a second strategic player inside the representative agent's optimization problem. Rather, it is a dummy draw from the current cross-sectional distribution. Formally, if $\tilde{X}_t \sim \mu_t$ and $\tilde{b}_t = b(t, \tilde{X}_t, \mu_t)$, then

$$\mathbb{E}[\partial_\mu V(t, x, \mu_t)(\tilde{X}_t) \tilde{b}_t] = \int \partial_\mu V(t, x, \mu_t)(y) b(t, y, \mu_t) \mu_t(dy).$$

Thus, the expectation is simply the integral of the measure derivative, weighted by the drift of agents at each state y , over the current population distribution.

For example, in the diffusion-only case

$$dX_t = b_t dt + \sigma_t dB_t,$$

a simplified Itô–Lions formula for $V(t, X_t, \mu_t)$ has the structure¹

$$\begin{aligned} dV(t, X_t, \mu_t) = & \partial_t V dt + \partial_x V dX_t + \frac{1}{2} \partial_{xx} V \sigma_t^2 dt \\ & + \mathbb{E} \left[\partial_\mu V(t, X_t, \mu_t)(\tilde{X}_t) \tilde{b}_t \right] dt + \frac{1}{2} \mathbb{E} \left[\partial_y (\partial_\mu V)(t, X_t, \mu_t)(\tilde{X}_t) \tilde{\sigma}_t^2 \right] dt \\ & + \text{local martingale terms.} \end{aligned} \quad (2.3)$$

where $\tilde{b}_t = b(t, \tilde{X}_t, \mu_t)$ and $\tilde{\sigma}_t = \sigma(t, \tilde{X}_t, \mu_t)$ denote the drift and volatility of the independent copy. The first line contains the standard Itô terms: the agent’s own state moves. The second line contains the mean-field terms: the population distribution moves. The independent-copy notation is what converts those distributional movements into expectations over population states. This is the technical step that turns the usual HJB equation into a master equation in (t, x, μ) .

The master equation can therefore be read as the dynamic-programming equation associated with the full mean-field state. Schematically, it takes the form

$$\begin{aligned} 0 = & \partial_t V(t, x, \mu) + \sup_a \{ r(t, x, \mu, a) + \mathcal{L}_x^{a, \mu} V(t, x, \mu) \} \\ & + \mathbb{E} \left[\partial_\mu V(t, x, \mu)(\tilde{X}) \tilde{b}(t, \tilde{X}, \mu, a^*(t, \tilde{X}, \mu)) \right] \\ & + \frac{1}{2} \mathbb{E} \left[\partial_y (\partial_\mu V)(t, x, \mu)(\tilde{X}) \tilde{\sigma}^2(t, \tilde{X}, \mu, a^*(t, \tilde{X}, \mu)) \right], \end{aligned} \quad (2.4)$$

up to the additional terms required by jumps, reflection, or common noise. If the state process has jumps, the generator also includes the usual compensator terms comparing post-jump and pre-jump values. If the state is reflected at a boundary, the Itô formula includes the corresponding boundary term, for example a term proportional to $\partial_x V$ evaluated at the reflecting boundary.

2.4. Connection with the limit order book application

In the application below, the agents are liquidity providers the book. Their state is their posted liquidity, and their control captures the intensity with which they replenish or adjust that liquidity. The strategic environment is summarized by the distribution of liquidity across providers. This distribution matters because execution risk and relative position in the book depend on where a trader stands in the cross-section. A trader with high posted liquidity is in a different strategic position from a trader with little liquidity, not because her identity matters, but because her rank in the distribution matters.

This is precisely the type of environment for which MFGs are useful. The model allows the order book to emerge as an equilibrium distribution rather than being imposed exogenously. It also gives a natural simulation strategy: solve the representative liquidity provider’s problem for a candidate distribution, update the implied distribution of liquidity, and iterate until the individual policy and the population law are mutually consistent. The next section specializes this general logic to limit order book dynamics and then introduces a rank-based simplification that makes the equilibrium computation tractable.

¹When the state process also includes jumps or reflection at a boundary, the same formula contains additional compensator terms for jumps and boundary/reflection terms associated with the regulating process.

2.5. Itô–Lions Lemma with Quantile Dependence

For our application to LOB dynamics, we assume that agent value functions depend on other agents' distribution μ only through the quantile function, or *rank*. Specifically, given a distribution μ with CDF F_μ , define the rank of an agent with liquidity state q as

$$u = F_\mu(q) \in [0, 1].$$

Intuitively, u is the fraction of agents with liquidity weakly below q . Larger u therefore means that the agent is higher in the liquidity distribution. If F_μ is strictly increasing, or if we regularize the distribution on a grid, define the quantile function

$$Q_\mu(u) = F_\mu^{-1}(u), \quad u \in [0, 1].$$

The quantile function is a convenient way to represent the entire distribution. Specifying Q_μ is equivalent to specifying μ : instead of indexing agents by their liquidity state q , one can index the population by ranks u and recover the liquidity of the agent at rank u from $Q_\mu(u)$.

This representation also clarifies how the distributional derivative in the Itô–Lions formula simplifies in the rank-based model. Suppose first that the representative value can be written as

$$V(t, q, \mu) = v(t, q, F_\mu(q)).$$

Thus, the distribution affects the value only through the rank $u = F_\mu(q)$. To see how a perturbation of the distribution affects the value, consider a nearby distribution μ' and the mixture path $\mu^p = (1 - p)\mu + p\mu'$, $p \in [0, 1]$. Along this path, the rank of an agent with liquidity q is

$$u^p = F_{\mu^p}(q) = (1 - p)F_\mu(q) + pF_{\mu'}(q).$$

Therefore,

$$\frac{d}{dp}u^p = F_{\mu'}(q) - F_\mu(q) = \int \mathbf{1}_{\{y \leq q\}} (\mu' - \mu)(dy).$$

At first order, the induced change in the value function is

$$V(t, q, \mu') - V(t, q, \mu) \approx v_u(t, q, u) \int \mathbf{1}_{\{y \leq q\}} (\mu' - \mu)(dy).$$

Hence, in the rank-based formulation, a perturbation of the distribution matters because it changes the agent's percentile position. Mass shifted below q increases the rank of an agent located at q , while mass shifted above q does not affect that rank directly.

Equivalently, one can work with the quantile representation. Fix a rank u and let $q = Q_\mu(u)$. The defining identity of the quantile is $F_\mu(Q_\mu(u)) = u$. Perturbing the distribution from μ to μ' along $\mu^p = (1 - p)\mu + p\mu'$ gives $F_{\mu^p}(Q_{\mu^p}(u)) = u$. Differentiating this identity with respect to p yields

$$f_{\mu^p}(Q_{\mu^p}(u)) \frac{d}{dp}Q_{\mu^p}(u) + \int \mathbf{1}_{\{y \leq Q_{\mu^p}(u)\}} (\mu' - \mu)(dy) = 0,$$

where f_{μ^p} is the density associated with F_{μ^p} . Therefore,

$$\frac{d}{dp} Q_{\mu^p}(u) = - \frac{\int \mathbf{1}_{\{y \leq Q_{\mu^p}(u)\}} (\mu' - \mu)(dy)}{f_{\mu^p}(Q_{\mu^p}(u))}.$$

This formula is the quantile analogue of the measure derivative. It says that the liquidity level associated with a fixed rank u changes when the distribution is perturbed, and the size of this change is scaled by the local density at that quantile. When the density is high, a small redistribution of mass has a small effect on the quantile; when the density is low, the same redistribution can move the quantile more strongly.

The same logic appears in the Itô–Lions formula. If the value depends on the distribution only through the rank,

$$V(t, q, \mu) = v(t, q, F_\mu(q)),$$

then the mean-field drift term can be written heuristically as

$$\mathbb{E} \left[\partial_\mu V(t, q, \mu_t)(\tilde{X}_t) \tilde{b}_t \right] = v_u(t, q, u) \int_{-\infty}^q b(t, y, \mu_t) \mu_t(dy), \quad u = F_{\mu_t}(q),$$

where $\tilde{X}_t \sim \mu_t$ is an independent copy and $\tilde{b}_t = b(t, \tilde{X}_t, \mu_t)$. The independent copy is only a compact way to write a population average. The expression says that the distributional drift of the value comes from the movement of agents located below q , since those are the agents whose motion changes the rank of an agent at q .²

3. A Model of Limit-Order-Book Dynamics

In this section we set up the main part of the paper: the model. We begin by specifying the mechanics of the asset whose underlying LOB is being constructed,

3.1. The Asset

We treat a generic asset valued at fixed exogenous fundamental price $p \in [P_{\min}, P_{\max}]$. We normalize this price to be in $[0, 1]$ ³:

$$\bar{p} = \frac{p - P_{\min}}{P_{\max} - P_{\min}}$$

A trader can buy or sell this asset through a market or limit order, the mechanics of which will now be discussed.

3.2. Transaction Cost

For each trade, one incurs a proportional transaction cost $f \in [0, 1]$; typical values are $f \in [0.01, 0.02]$. An LP posting at ask a receives $a(1 - f)$ upon the order being filled. This affects the LP's profit per fill and introduces a no-arbitrage floor on admissible asks.⁴

²To avoid using the indicator in the CDF, one can replace the rank by a smooth approximation $u^h(q, \mu) = \int K_h(q - y) \mu(dy)$ where K_h is a smooth approximation of the indicator $\mathbf{1}_{\{y \leq q\}}$. Then $V(t, q, \mu) = v(t, q, u^h(q, \mu))$ and the corresponding mean-field drift term becomes $\mathbb{E} \left[\partial_\mu V(t, q, \mu_t)(\tilde{X}_t) \tilde{b}_t \right] = v_u(t, q, u^h) \int K_h(q - y) b(t, y, \mu_t) \mu_t(dy)$.

³For the remainder of the paper, we assume p is always normalized to the interval $[0, 1]$.

⁴In our experiment we set the transaction fees to 0.

3.3. Trader Population

A continuum of traders with total mass $N \in \mathbb{N}^*$ arrives according to a Poisson process with rate $\lambda \in \mathbb{R}_{\geq 0}$. Upon arrival, each trader draws a type (v, r) :

- **Valuation** $v \sim F(v)$ on $[v_{\min}, v_{\max}]$: the trader's subjective valuation of the asset.
- **Discount rate** $r \sim G(r)$ on $[r_{\min}, r_{\max}]$: a proxy for impatience or cost of capital, capturing commitment to belief.

3.4. Trader Classification

Given the LOB's best ask a_{best} and best bid b_{best} , a trader of type v is classified as follows.

Upon arrival, a trader with valuation v :

- submits a **market buy** if $v \geq a_{\text{best}}$, consuming δ_b shares from the ask;
- submits a **market sell** if $v \leq b_{\text{best}}$, consuming δ_b shares from the bid;
- posts a **limit order** (becomes an LP) if $v \in (b_{\text{best}}, a_{\text{best}})$. The LP posts on the ask side if $v < p$ and on the bid side if $v > p$.

Remark 3.1 (On the $v \lessgtr p$ rule). Strictly, a trader with $v \in (b_{\text{best}}, a_{\text{best}})$ ought to choose the side that maximises their expected discounted profit:

$$\max \left(\max_{a \geq p} V_0^{\text{ask}}(v, r, a), \max_{b \leq p} V_0^{\text{bid}}(v, r, b) \right).$$

For narrow spreads, this reduces to the simple rule $v \lessgtr p$, which we adopt as a modelling convention. The rule is consistent with full optimisation provided $|v - p|$ dominates the spread for the majority of LPs which is a condition satisfied in our numerical experiments.

3.5. Motivation for a Unified Population

The distribution F simultaneously determines three distinct economic quantities:

- **Market order flow**: $\phi(a) = \lambda[1 - F(a)]$.
- **LP cost bases**: each LP's valuation v is their effective cost basis, yielding margin $a(1 - f) - v$ per fill.
- **Role classification**: the classification in Section 3.4.

This coupling through a single primitive is a conceptual advance over frameworks with independent buyer and LP populations: belief heterogeneity alone explains demand, supply, and the buyer/seller/LP boundary.

3.6. Input Parameters

Table 1. Summary of model input parameters and their descriptions.

Group	Parameter	Description
Market	p	Normalised mid / fair value of the asset
	f	Proportional transaction cost
	λ	Poisson arrival rate of traders
	N	Total mass of the trader continuum
Distributions	$F(v)$	Valuation Distribution
	$G(r)$	LP discount rate distribution
Inventory	κ	Quadratic replenishment cost coefficient
	$\bar{\ell}$	Maximum replenishment intensity
	$\delta_s = \delta_b$	Jump size (replenishment / execution unit)
	ψ	Replenishment rank-coupling weight
	θ	Execution rank-coupling weight
Grids	N_v	Number of valuation grid points
	N_r	Number of discount-rate grid points
	N_a	Number of candidate price levels
	N_q	Number of queue-depth states
	$q_{\max}$	Queue-depth truncation level
Solver	ω	Outer-loop Picard damping factor
	T_{outer}	Number of Bertrand (outer) iterations
	T_{Cesaro}	Outer iteration at which Cesàro averaging starts
	σ	Gaussian aggregation kernel bandwidth

Building on the primitives of Section 3, we now set the mean-field game machinery.

4. Applied Mean-Field Game

Let $q_t \in \mathbb{R}_+$ denote an individual seller's liquidity. The population distribution of states at time t is a probability measure $\mu_t \in \mathcal{P}(\mathbb{R}_+)$. The representative seller chooses a control $l_t \in [0, \bar{l}]$ (interpretable as posting or replenishment intensity).

Liquidity evolves via two jump processes: upward replenishments and downward executions. We impose a reflection at 0 to ensure $q_t \geq 0$ (as liquidity cannot be negative). In their simplest form the dynamics are:

$$dq_t = \delta_s dN_t^s - \delta_b dN_t^b + dK_t, \quad q_t \geq 0 \quad (4.1)$$

where $\delta_s, \delta_b > 0$ are jump sizes, N^b, N^s are counting processes with possibly state or law dependent intensities, and K_t is a minimal reflection process that prevents negative liquidity. In discrete state implementations, reflection is equivalent to mapping any transition $q \mapsto q - \delta_b$ to 0.

In a fully general MFG, intensities and payoffs may depend on μ_t in arbitrary ways, thus:

$$\lambda_s(q, \mu, l), \quad \lambda_b(q, \mu), \quad \pi(q, \mu)$$

The equilibrium requirement is a fixed point: the distribution μ_t used by the representative agent to optimize must coincide with the distribution generated when everyone uses the equilibrium control.

For our application, we assume replenishment and consumption intensities depend on μ only through the rank:

$$\lambda_s(q, \mu, l) = \phi_s(q, l)\Psi(F_\mu(q)), \quad \lambda_b(q, \mu) = \phi_b(q)\Theta(F_\mu(q)). \quad (4.2)$$

The functions $\Psi, \Theta : [0, 1] \rightarrow \mathbb{R}_+$ are distributional couplings that encode how competition and execution pressure vary across ranks.

For a first implementation, we recommend bounded linear forms:

$$\Psi(u) = 1 + \psi u, \quad \psi \geq 0, \quad \Theta(u) = 1 + \theta(1 - u), \quad \theta \geq 0. \quad (4.3)$$

These can be interpreted as replenishment becoming easier at higher ranks due to scale, while execution pressure is stronger at lower ranks, closer to the top of the book.

4.1. Trader Value Function and HJB Equation

We now use this rank based coupling and introduce controls and costs to construct the HJB. Then we consider the Kolmogorov Forward Equation KFE to reconcile the HJB with the mean-field equilibrium.

For the control and costs we choose a standard linear-quadratic structure:

$$\phi_s(q, l) = a(q)l, \quad c(l) = \frac{\kappa}{2}l^2, \quad l \in [0, \bar{l}] \quad (4.4)$$

where $a(q) \geq 0$ captures how effective posting intensity is at state q , and $\kappa > 0$ scales posting cost.

For the payoff, we interpret downward jumps as execution events. A simple per-unit margin specification is:

$$\pi(q, \mu) = m\phi_b(q)\Theta(F_\mu(q))\delta_b\mathbf{1}\{q > 0\} \quad (4.5)$$

where $m > 0$ is a per-unit profit margin.

We focus on an infinite-horizon stationary equilibrium. Let $\rho > 0$ be the discount rate. Given a candidate stationary distribution μ (equivalently, its CDF F_μ or quantile Q_μ), the representative seller solves:

$$V(q; \mu) = \sup_{l_t \in [0, \bar{l}]} \mathbb{E} \left[\int_0^\infty e^{-\rho t} (\pi(q_t; \mu) - c(l_t)) \mid q_0 = q \right]$$

where q_t follows (4.1) with intensities (4.2).

Under standard dynamic programming arguments for controlled jump processes, $V(\cdot; \mu)$ satisfies:

$$\begin{aligned} rV(q; \mu) = & \max_{l \in [0, \bar{l}]} \{ \pi(q; \mu) - c(l) \\ & + \lambda_s(q, \mu, l) [V(q + \delta_s; \mu) - V(q; \mu)] \\ & + \lambda_b(q, \mu) [V((q - \delta_b)^+; \mu) - V(q; \mu)] \} \end{aligned} \quad (4.6)$$

With reflection entering through $(q - \delta_b)^+$.

With (4.4), (4.6) becomes a quadratic maximization problem, thus yielding the closed form best response (Ma and Noh, 2020):

$$l^*(q; \mu) = \prod_{[0, \bar{l}]} \left(\frac{a(q)\Psi(F_\mu(q))}{\kappa} [V(q + \delta_s; \mu) - V(q; \mu)] \right) \quad (4.7)$$

where $\prod_{[0, \bar{l}]}$ is the projection onto the interval $[0, \bar{l}]$.

From here we can see the key simplification: the best response can be computed pointwise once V is known.

4.2. Kolmogorov Forward Equation and Mean-Field Nash Equilibrium

Given a feedback policy $l^*(\cdot, \mu)$, the stationary distribution μ satisfies:

$$\int \mathcal{A}^\mu f(q) \mu(dq) = 0 \text{ for all bounded test functions } f \quad (4.8)$$

with controlled generator:

$$\mathcal{A}^\mu f(q) = \lambda_s(q, \mu, l^*(q; \mu)) [f(q + \delta_s) - f(q)] + \lambda_b(q, \mu) [f((q - \delta_b)^+) - f(q)] \quad (4.9)$$

A stationary mean-field Nash equilibrium is a probability measure μ^* and associated value and optimal control (V^*, l^*) such that:

1. Given μ^* , $V^*(\cdot; \mu^*)$ solves (4.6) and $l^*(\cdot; \mu^*)$ is given by (4.7)
2. Under policy $l^*(\cdot; \mu^*)$, the reflected jump process has stationary law μ^* , i.e. (4.8) holds with $\mu = \mu^*$

This can be formulated equivalently with quantiles. μ^* is a fixed point of the map $Q \rightarrow \tilde{Q}$, where $\tilde{Q}$ is the quantile function of the stationary distribution generated by the HJB solution under Q . This reformulation in terms of quantiles is the basis of the computational algorithm developed in Section 5.

4.3. Equilibrium Conditions

Stated formally, an equilibrium is a tuple

$$(d^*, \{a_{k,l}^*\}, \{\bar{d}_{k,l}\}, \{V_{k,l}^*\}, \{l_{k,l}^*\}, \{m_{k,l}^*\})$$

such that, for every LP type (v_k, r_l) :

1. **(HJB)** $V_{k,l}^*$ satisfies the stationary HJB given $l_{k,l}^*$ and $\mu^*(a_{k,l}^*) = \phi(a_{k,l}^*)/\mathcal{D}^*(a_{k,l}^*)$;
2. **(FOC)** $l_{k,l}^*$ satisfies the policy FOC given $V_{k,l}^*$;
3. **(KFE)** $m_{k,l}^*$ is the stationary distribution of the Markov chain induced by $l_{k,l}^*$;
4. **(Bertrand)** $a_{k,l}^* \in \arg \max_{a \geq p} V_0(v_k, r_l, a; d^*)$;
5. **(Aggregation)** $d^*(a) = N \sum_{k,l} f(v_k) g(r_l) w_k^v w_l^r \bar{d}_{k,l} K_\sigma(a - a_{k,l}^*)$, with appropriate normalization of the Gaussian kernel K_σ .

We will construct such a tuple in the algorithm in Section 5 by Picard iteration. We do not establish existence or uniqueness analytically; convergence is observed empirically in a Cesaro / mixed-strategy sense.

5. Algorithm

This section provides an algorithm that iterates directly over the quantile function Q_μ to sidestep the more complicated measure-derivative machinery. As discussed in Section 2.5 once Q_μ is found, it is possible to recover the stationary distribution, allowing us to compute the shape of the LOB.

We will begin by detailing what problem the algorithm is actually trying to solve, then providing the full algorithm.

5.1. Problem Setting

We seek the equilibrium ask-side limit order book, characterized by:

- A density function $d : [p, 1 - f] \rightarrow \mathbb{R}_{\geq 0}$, where $d(a)da$ is the equilibrium mass of posted depth in an infinitesimal price interval around a .
- A corresponding cumulative depth $\mathcal{D}_{\text{cum}} \int_{a_{\text{best}}}^a d(s)ds$.

For the full LOB, the density and cumulative depth on the bid side are obtained by running the same algorithm and reflecting.

On the bid side the cumulative depth is measured symmetrically, from the best bid inward: $\mathcal{D}_{\text{cum}}(b) = \int_b^{b_{\text{best}}} d_{\text{bid}}(s) ds$.

Density $d(a)$ is not a primitive of the model. It is the aggregate of three coupled equilibria:

1. A within LP optimization. Each LP type posting at ask a chooses a replenishment policy $\{l\}_{n=0}^{N_q}$ to maximize expected discounted profit given depth dynamics. This is a stochastic control problem as the LP's value function V_n satisfies the HJB.
2. A within depth stationary distribution condition. Given the optimal policy, the LP's depth process is a continuous-time Markov chain with a stationary distribution m_0 . The HJB itself depends on this distribution through the rank coupling functions. So V , l^* , and m_0 must be mutually consistent.
3. A cross-price Bertrand competition. Each LP type chooses its posting price a to maximize $V_0(v, r, a)$. The optimum depends on the fill rate $\mu_0(a) = \frac{\phi(a)}{\mathcal{D}(a)}$, which depends on the aggregate depth $\mathcal{D}(a)$ from all LP types. The Bertrand best response and the aggregate depth must be mutually consistent.

We thus have the following dependency for the above conditions:

$$V \xrightarrow{\text{FOC}} l^* \xrightarrow{\text{Markov chain}} m_0 \xrightarrow{\text{rank } u} V$$

There is no closed form for the output elements. The algorithm retrieves them through a nested sequence of fixed point iterations.

The goal is to reach the following outputs:

Table 2. Summary of outputs of the algorithm.

Symbol	Meaning
$d^*(a)$	Equilibrium ask-side density
$\mathcal{D}_{\text{cum}}^*(a)$	Equilibrium cumulative ask depth
$a^*(v, r)$	LP's optimal ask given type
$\bar{d}(v, r)$	LP's equilibrium mean depth at the optimal ask
$V_0(v, r)$	LP's equilibrium expected discounted profit at zero inventory
a^{best}	The best ask in the equilibrium LOB
$\mu_0^*(a) = \phi(a)/\mathcal{D}^*(a)$	Equilibrium pro-rata fill rate

5.2. Architecture

A direct approach on all five equilibrium conditions simultaneously is intractable. Instead, we exploit the dependencies hierarchical nature through a nested loop approach.

- The HJB and FOC conditions involved only one of (v, r, a) at a time. These are handled by an inner loop, which alternates value evaluation and policy improvement until V and l^* are consistent.
- The KFE condition couples to the HJB only through the within-price rank coupling. In our implementation, u is fixed and the middle loop runs a short fixed step smoother that updates the grid $\{q_n\}$ given the latest stationary distribution. The middle loop is not a converged solve in our implementation, it is a damped smoother whose remaining error is absorbed by the outer loop averaging.
- The Bertrand and aggregation conditions couple every LP type to every other through $\mathcal{D}(a)$. These are handled by the outer loop, which iterates on $\mathcal{D}(a)$ through damped Picard and Cesaro averaging.

The equilibrium consists of three nested fixed-point computations:

Table 3. Three-loop architecture of the equilibrium algorithm.

Level	Object	Method	Iterations
Outer	Aggregate depth $\mathcal{D}(a)$	Damped Picard + Cesaro mean	$T_{\text{outer}} = 18$
Middle	Quantile function $Q(u)$	Damped iteration, fixed count	$T_{\text{in}} \in \{3, 5\}$
Inner	Value function V and policy l^*	Policy iteration	up to $T_{\text{PI}} = 80$

Information flows downward as parameters and upward as solutions. The outer loop fixes $\mathcal{D}(a)$, from which it derives $\mu_0(a) = \frac{\phi(a)}{\mathcal{D}(a)}$. The middle loop receives $\mu_0(a)$ at each candidate price level

and an LP type (v, r) . The inner loop receives the depth grid and the rates derived from μ_0 . The inner loop returns V and l^* to the middle, which returns V_0 and $\bar{d}$ to the outer.

5.3. Rank Grid

The natural choice for the rank grid is a fixed depth one $q_n = n\delta$ with $n = 0, 1, 2, \dots, (N_q - 1)$. However, at each iteration, the stationary distribution m changes, so the ranks $u_n = \sum_{k \leq n} m_k$ change too. Thus $\Psi(u), \Theta(u)$ in the HJB become moving targets, making convergence fragile.

The alternative is to use a fixed rank grid, which is more stable. Define a uniform rank grid:

$$u_n = \frac{n - \frac{1}{2}}{N_q}, \quad n = 1, 2, \dots, N_q$$

This grid uses the midpoint convention, thus avoiding the endpoints $u = 0, 1$ where the rank coupling functions would degenerate.

The unknowns become the depth values at each rank: the quantile function $Q(u_n) = q_n$. So instead of solving for the distribution of depth at fixed depths, we solve for the depth at fixed quantiles. The coupling functions evaluated at a specific point on the grid thus become constants of the iteration, improving convergence.

5.4. Inner Loop: Policy Iteration

Algorithm 1 Policy Iteration

Input: Depth grid $\{q_n\}$, rank grid $\{u_n\}$, jump destinations $\{j^+(n), j^-(n)\}$, discount rate r , margin $m(a, v) = a(1 - f) - v$, fill rate μ_0 , parameters $(\kappa, \bar{l}, \psi, \theta, \delta_b)$

Output: Value function V^* , optimal policy l^*

$l_n^{(0)} \leftarrow 0, V_n^{(0)} \leftarrow 0$ for all n

for $k = 0, 1, \dots, T_{PI} = 80$ **do**

 Compute rates:

$\lambda_s^n \leftarrow l_n^{(k)} \Psi(u_n), \quad \lambda_b^n \leftarrow \mu_0 \Theta(u_n)$

 Build linear system $AV = b$:

$A_{nn} \leftarrow r + \lambda_s^n + \lambda_b^n, \quad A_{n, j^+(n)} \leftarrow -\lambda_s^n, \quad A_{n, j^-(n)} \leftarrow -\lambda_b^n$

$b_n \leftarrow m \delta_b \lambda_b^n \mathbf{1}_{n \geq 1} - \frac{\kappa}{2} (l_n^{(k)})^2$

 Solve $V^{(k+1)} \leftarrow A^{-1}b$ via `np.linalg.solve`

 Update policy via FOC:

$l_n^{(k+1)} \leftarrow \Pi_{[0, \bar{l}]} \left(\frac{\Psi(u_n)}{\kappa} [V_{j^+(n)}^{(k+1)} - V_n^{(k+1)}] \right)$

if $\|l^{(k+1)} - l^{(k)}\|_\infty < 10^{-10}$ **then**

break

return $V^{(k+1)}, l^{(k+1)}$

Interpretation. the LPs take fill rate $\mu_0(a)$ as a given and solve the optimal control problem for the replenishment intensity, there is no strategic interaction yet, just optimizing internally within-period.

5.5. Middle Loop: Quantile Iteration

Algorithm 2 Quantile Fixed Point

Input: LP type (v, r) , candidate ask a , fill rate $\mu_0 = \mu_0(a)$, iteration count T_{in}

Output: $V_0 = V_0^{(T_{\text{in}})}$, mean depth $\bar{d} = \delta_b \sum_n n \cdot m_n^{(T_{\text{in}})}$

Initialise depth grid: $q_n^{(0)} \leftarrow (n-1) \cdot q_{\text{max}} / (N_q - 1)$ for $n = 1, \dots, N_q$, with $q_{\text{max}} = 20$

for $t = 1, \dots, T_{\text{in}}$ **do**

 Compute jump destinations:

$$j^+(n) \leftarrow \arg \min_k |q_k^{(t-1)} - (q_n^{(t-1)} + \delta_s)|, \quad j^-(n) \leftarrow \arg \min_k |q_k^{(t-1)} - \max(q_n^{(t-1)} - \delta_b, 0)|$$

▷ Destinations updated each iteration as the depth grid evolves

 Call Algorithm 1 to obtain $V^{(t)}, l^{*(t)}$

 Compute optimal rates and dominating rate:

$$\lambda_s^n \leftarrow l_n^{*(t)} \Psi(u_n), \quad \lambda_b^n \leftarrow \mu_0 \Theta(u_n), \quad \bar{\Lambda} \leftarrow \max_n (\lambda_s^n + \lambda_b^n) + 10^{-12}$$

 Build uniformised transition matrix:

$$P_{n, j^+(n)} \leftarrow \frac{\lambda_s^n}{\bar{\Lambda}}, \quad P_{n, j^-(n)} \leftarrow \frac{\lambda_b^n}{\bar{\Lambda}}, \quad P_{nn} \leftarrow 1 - \frac{\lambda_s^n + \lambda_b^n}{\bar{\Lambda}}$$

▷ Stationary distribution of P equals that of the continuous-time chain

 Initialise $m_0 \leftarrow \mathbf{1}/N_q$, $m_{\text{prev}} \leftarrow \mathbf{0}$

while $\|m_0 - m_{\text{prev}}\|_{\infty} \geq 10^{-14}$ **do**

$m_{\text{prev}} \leftarrow m_0$
 $m_0 \leftarrow P^{\top} m_0 / \|P^{\top} m_0\|_1$

 Compute CDF: $M_n \leftarrow \sum_{k \leq n} m_k^{(t)}$

if $M_n \leq M_{n-1}$ **then**

$M_n \leftarrow M_{n-1} + 10^{-12}$ *▷ Monotonicity safeguard against degenerate CDF inversion*

 Interpolate new quantile function:

$$\tilde{Q}^{(t)}(u_n) \leftarrow \text{interp}(u_n; M, q^{(t-1)})$$

▷ Piecewise linear; boundary extrapolation to $q_1^{(t-1)}$ and $q_{N_q}^{(t-1)}$

 Damp update:

$$q_n^{(t)} \leftarrow (1 - \omega_{\text{in}}) q_n^{(t-1)} + \omega_{\text{in}} \tilde{Q}^{(t)}(u_n), \quad \omega_{\text{in}} = 0.15$$

 Enforce monotonicity: $q^{(t)} \leftarrow \text{sort}(q^{(t)})$

 Enforce non-negativity: $q^{(t)} \leftarrow \max(q^{(t)}, 0)$

return $V_0^{(T_{\text{in}})}$, $\bar{d} = \delta_b \sum_n n \cdot m_n^{(T_{\text{in}})}$

Interpretation. This loop takes the optimal control as given and computes the average order queue over time, deducing the long run value from posting at the candidate ask.

Note on T_{in} . We use $T_{\text{in}} = 3$ during early outer iterations (when $\mu_0(a)$ is far from its equilibrium value) and $T_{\text{in}} = 5$ after the tenth outer iteration. The middle loop is therefore not run to convergence; it functions as a fixed-step smoother that amortises convergence over the outer Bertrand iterations. This is a practical compromise — running the middle loop to tight convergence at every Bertrand evaluation would increase runtime by an order of magnitude with marginal benefit, since the outer damping already smooths residual error.

5.6. Outer Loop: Bertrand Fixed Point

Algorithm 3 Bertrand Fixed Point

Input: Beta parameters (α_F, β_F) , fundamental p , fee f , primitives $(\lambda, \kappa, \bar{l}, \psi, \theta, N, r_{\min}, r_{\max})$

Output: Equilibrium density $d^*(a) = \bar{\mathcal{D}}(a)$, cumulative depth $\mathcal{D}_{\text{cum}}^*(a) = \int_{a_{\text{best}}}^a d^*(s) ds$

Initialise $\mathcal{D}^{(0)}(a_j) \leftarrow 2.0$ for all j

Initialise $\bar{\mathcal{D}} \leftarrow 0$, $n_{\text{cesaro}} \leftarrow 0$

for $t = 0, 1, \dots, T_{\text{outer}} - 1$ **do**

 Compute fill rates:

$$\mu_0^{(t)}(a_j) \leftarrow \frac{\phi(a_j)}{\max(\mathcal{D}^{(t)}(a_j), 0.01)}, \quad \phi(a) = \lambda[1 - F_{\text{Beta}}(a; \alpha_F, \beta_F)]$$

for each LP type (v_k, r_l) **do**

for each admissible a_j s.t. $a_j(1 - f) - v_k > 5 \times 10^{-4}$ **do**

 Call Algorithm 2 to obtain $V_0(v_k, r_l, a_j)$ and $\bar{d}(v_k, r_l, a_j)$

 Bertrand choice:

$$j^*(k, l) \leftarrow \arg \max_j V_0(v_k, r_l, a_j), \quad a_{k,l}^* \leftarrow a_{j^*}, \quad \bar{d}_{k,l} \leftarrow \bar{d}(v_k, r_l, a_{j^*})$$

 Aggregate with Gaussian kernel smoothing:

$$\mathcal{D}^{\text{new}}(a_j) \leftarrow \sum_{k,l} N \cdot f_{\text{Beta}}(v_k) \cdot g(r_l) \cdot w_k^v \cdot w_l^r \cdot \bar{d}_{k,l} \cdot \frac{K_\sigma(a_j - a_{k,l}^*)}{\sum_{j'} K_\sigma(a_{j'} - a_{k,l}^*) \cdot \Delta a_{j'}}$$

▷ $K_\sigma(x) = \exp(-x^2/2\sigma^2)$, $\sigma = 0.015$; normalisation ensures each type contributes full $\bar{d}_{k,l}$ in expectation

 Clamp: $\mathcal{D}^{\text{new}} \leftarrow \max(\mathcal{D}^{\text{new}}, 0.005)$

 Damp update:

$$\mathcal{D}^{(t+1)} \leftarrow (1 - \omega) \mathcal{D}^{(t)} + \omega \mathcal{D}^{\text{new}}$$

if $t \geq T_{\text{cesaro}} = 11$ **then**

$$\begin{aligned} n_{\text{cesaro}} &\leftarrow n_{\text{cesaro}} + 1 \\ \bar{\mathcal{D}} &\leftarrow \bar{\mathcal{D}} + \frac{\mathcal{D}^{(t+1)} - \bar{\mathcal{D}}}{n_{\text{cesaro}}} \end{aligned}$$

▷ Online Cesaro mean over iterations $t \geq T_{\text{cesaro}}$

return $d^*(a) = \bar{\mathcal{D}}(a)$, $\mathcal{D}_{\text{cum}}^*(a) = \int_{a_{\text{best}}}^a d^*(s) ds$ by trapezoidal rule

Interpretation. This is the Bertrand competition layer, it answers the question of what is the best price to post at given what every other LP is doing. Furthermore, it checks that everyone is consistent with the market depth that makes those choices rational.

Note on Cesaro Averaging. The outer iteration does not converge pointwise in our experiments. The point sequence $\mathcal{D}^{(t)}$ settles into a bounded cycle: $a_{k,l}^*$ flips between two or three adjacent grid points each iteration, and the aggregate $\mathcal{D}$ wobbles correspondingly. This is a generic feature of Bertrand games on discrete grids, the best response is integer valued and cannot be approached smoothly.

The Cesaro mean corresponds to the average behaviour over the cycle, equivalent to a mixed strategy over the adjacent best responses. For empirical applications, where one observes time-averaged LOB snapshots rather than instantaneous ones, this is the appropriate object. We do not claim formal convergence; the equilibrium exists in a Cesaro / mixed strategy sense.

6. Results

We run the model with the following parameter inputs:

Table 4. Model input parameters and their chosen values for the experiment.

Group	Parameter	Value
Market	p	0.65
	f	0.00
	λ	10.0
	N	100
Distributions	$F(v)$	Beta(6.5, 3.5)
	$G(r)$	Uniform(0.05, 2.0)
Inventory	κ	0.4
	$\bar{\ell}$	3.0
	$\delta_s = \delta_b$	1.0
	ψ	0.3
	θ	0.7
Grids	N_v	6
	N_r	6
	N_a	14
	N_q	35
	$q_{\max}$	20.0
Solver	ω	0.10
	T_{outer}	18
	$T_{\text{Cesàro}}$	11
	σ	0.015

The numerical outputs are:

Table 5. Equilibrium outcomes of the order book for a general asset.

Quantity	Ask side	Bid side
Reference price p	0.650	0.350
Price floor	0.6501	0.3501
Best quote	0.6501	0.3501
Floor binds	yes	yes
Most aggressive $a_{\min}^*$	0.6501	0.3890
Least aggressive $a_{\max}^*$	0.8828	0.6049
LP valuation support	[0.344, 0.645]	[0.106, 0.345]
Total book depth	11.00	12.91
<i>Combined book</i>		
Best ask	0.6501	
Best bid	0.6499	
Implied mid	0.6500	
Spread s	0.0002	

6.1. Densities and Book Shape

We first look at the primary output of interest: the LOB shape and the densities of the orders at each price level.

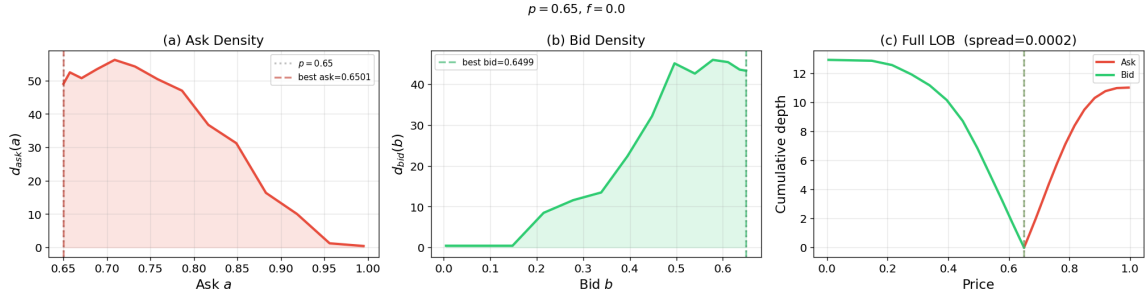


Figure 1. Order densities and LOB shape.

We can see that the algorithm successfully produced a shape resembling that of a LOB. In particular, we see that orders tend to pile up at price extremes; it is consistent with the fact that more "extreme" orders are less likely to be executed thus they pile up and cause a higher density.

Another striking revelation is that the bid-ask spread pins on the no arbitrage floor, this is because every liquidity provider earns a positive margin even when quoting at the mid, so Bertrand competition drives both touches to p .

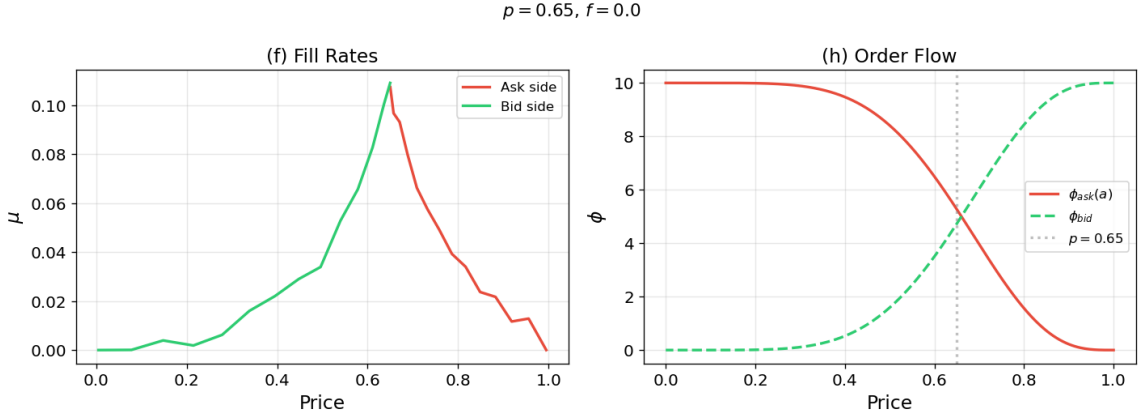


Figure 2. Order fill rates and flow.

The fill rate profile reinforces this reading. It is highest near the best ask/bid and decays away from the mid. Orders posted close to the touch are therefore executed quickly and do not accumulate, keeping depth thin at the frontier, while orders posted far from the mid are filled only rarely and build up, thus producing the high density at the price extremes seen in Figure 1.

Finally, we consider the Bertrand sorting panels, which tell a related story about how orders are distributed.

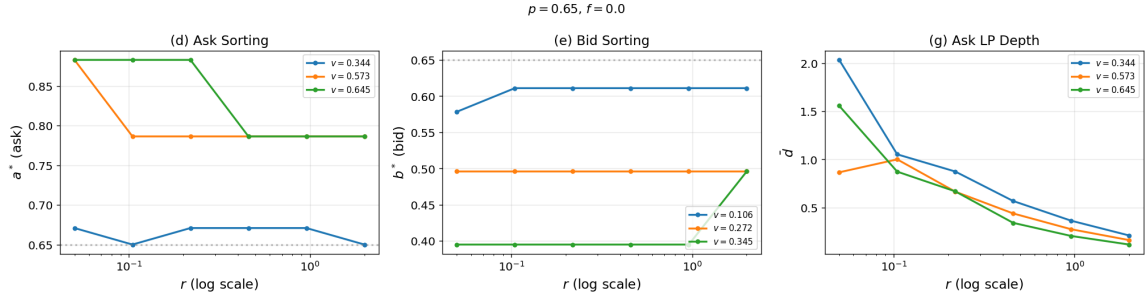


Figure 3. Bertrand Sorting and depth based on discount rate.

As expected, higher-valuation traders tend to post at higher prices on the ask side and lower prices on the bid side: an LP will not post below their own cost basis v , so the optimal quote is increasing in v . The more revealing pattern is the dependence on the discount rate. For a fixed valuation, the optimal ask $a^*(v, r)$ is decreasing in r : patient LPs post far from the mid, accepting a low fill probability in exchange for a larger margin per execution, while impatient LPs post close to the touch, trading margin for speed of execution. This is the Bertrand sorting the model produces, where liquidity providers separate across the book by patience, with the most impatient setting the frontier and the most patient supplying the deep liquidity. The mean-depth panel mirrors this: $\bar{d}$ declines in r , so impatient LPs not only post closer to the mid but also hold smaller inventories there, consistent with their quotes being consumed quickly.

7. Conclusion

We have built a Bertrand LOB mean-field game model, in which a single belief distribution simultaneously governs market order flow, LP cost bases, and the buyer/seller boundary. We used the fact that LP heterogeneity in valuation and discount rate enter as primitives, computing equilibrium via the quantile fixed point algorithm, resulting in a Bertrand sorting of traders. We then specialized it to a theoretical asset with fundamental price in the interval $[0, 1]$.

The model produces a full LOB shape derived from strategic primitives, without imposing any exogenous functional form on depth, essentially a book constructed "from scratch". While bid ask spread is mechanically endogenous, it isn't yet on the economic side, as it pins on the no-arbitrage floor rather than being competition determined. Furthermore, there are several natural extensions that propose themselves.

Extensions. A direct next step would be to calibrate the valuation distribution $F(v)$ to real data on a chosen asset (available publicly through APIs like Yahoo Finance) and upgrade discount rate distribution $G(r)$ to a calibrated Pareto distribution, allowing us to compare our modelled LOB to empirical observations at different fundamental prices.

Another important extension would be to ease the exogenous mid price assumption by implementing it through a stochastic process (for example, through a martingale driven by information arrival). This would add an important risk channel for the agents, who currently face execution uncertainty but no mark-to-market risk from holding a position in a dynamic market. Furthermore, if we give a fraction of buyers insider information about the market, traders would now face a [Glosten and Milgrom \(1985\)](#) adverse selection problem on top of the Bertrand competition, driving another risk channel.

Finally, a natural complement would be to introduce a designated market maker (leader) whose rules (incentives, fees, price schedules) the population of retail LPs (followers) responds to – a Stackelberg mean-field game in the sense of [Zeng et al. \(2025\)](#).

8. Appendix

8.1. Algorithm Code

```
1 # Required Imports
2 import os
3 os.environ["OMP_NUM_THREADS"] = "1"
4 os.environ["MKL_NUM_THREADS"] = "1"
5 os.environ["OPENBLAS_NUM_THREADS"] = "1"
6
7 import numpy as np
8 from scipy.stats import beta as beta_dist
9 import matplotlib.pyplot as plt
10 import time
11 from joblib import Parallel, delayed
12
13 # NumPy compatibility shim (np.trapz removed in NumPy 2.x)
14 if not hasattr(np, "trapz"):
15     np.trapz = np.trapezoid
16
17 plt.rcParams.update({
18     'figure.figsize': (10, 6), 'font.size': 12, 'axes.labelsize':
19     14,
20     'axes.titlesize': 14, 'legend.fontsize': 11, 'lines.linewidth':
21     2,
22     'figure.dpi': 120,
23 })
24
25 # Primitives
26 def proj_0_lbar(x, lbar):
27     return np.minimum(np.maximum(x, 0.0), lbar)
28
29 def rank_grid(N):
30     return (np.arange(1, N+1) - 0.5) / N
31
32 def destination_indices(q, ds, db):
33     q = np.asarray(q); N = q.size
34     jp = np.empty(N, dtype=int); jm = np.empty(N, dtype=int)
35     for i in range(N):
36         jp[i] = int(np.argmin(np.abs(q - (q[i] + ds))))
37         jm[i] = int(np.argmin(np.abs(q - max(q[i] - db, 0.0))))
38     return jp, jm
39
40 def solve_hjb(q, u, jp, jm, rho, kappa, l_bar, psi, theta, margin,
41     db,
42     mu_exec, max_iter=80, tol=1e-10):
43     N = len(q); Psi = 1+psi*u; Theta = 1+theta*(1-u)
```



```

41 l = np.zeros(N); V = np.zeros(N)
42 for it in range(max_iter):
43     ls = l*Psi; lb = mu_exec*Theta
44     pi = margin*lb*db*(q > 0).astype(float)
45     A = np.zeros((N, N)); b = np.zeros(N)
46     for i in range(N):
47         A[i,i] = rho + ls[i] + lb[i]
48         A[i,jp[i]] -= ls[i]; A[i,jm[i]] -= lb[i]
49         b[i] = pi[i] - 0.5*kappa*l[i]**2
50     try: Vn = np.linalg.solve(A, b)
51     except: break
52     ln = proj_0_lbar((Psi/kappa)*(Vn[jp]-Vn), l_bar)
53     if np.max(np.abs(ln-l)) < tol: V,l = Vn,ln; break
54     V,l = Vn,ln
55 return V, l
56
57 def stat_dist(jp, jm, lp, lm):
58     N = len(lp); rates = np.array(lp)+np.array(lm)
59     bL = max(np.max(rates), 1e-12); P = np.zeros((N,N))
60     for i in range(N):
61         if rates[i] > 0:
62             P[i,jp[i]] += lp[i]/bL; P[i,jm[i]] += lm[i]/bL
63             P[i,i] += 1-rates[i]/bL
64         else: P[i,i] = 1
65     m = np.ones(N)/N; PT = P.T
66     for _ in range(80000):
67         mn = PT@m; s = mn.sum()
68         if s <= 0: break
69         mn /= s
70         if np.max(np.abs(mn-m)) < 1e-14: m = mn; break
71     m = mn
72 return m
73
74 def quant_upd(u, q, m):
75     M = np.cumsum(m); M = np.clip(M, 0, 1); Me = M.copy()
76     for i in range(1, len(Me)):
77         if Me[i] <= Me[i-1]: Me[i] = Me[i-1]+1e-12
78     return np.interp(u, np.clip(Me, 0, 1), q, left=q[0], right=q
[-1])

```

```

1
2 # Grid Constructor
3 def beta_quantile_grid(alpha, beta, v_min_prob, v_max_prob, N):
4     """
5     Return N points spaced by Beta quantiles in probability interval
6     [v_min_prob, v_max_prob]. This puts more grid points where the
7     Beta density is higher, avoiding wasted resolution in low-

```

```

    density tails.
8     """
9     # Uniformly spaced probabilities mapped through the Beta inverse
    CDF
10    probs = np.linspace(v_min_prob, v_max_prob, N)
11    return beta_dist.ppf(probs, alpha, beta)
12
13 # One Side Solver
14 def _solve_type_pair(kv, kr, v_val, r_val, a_grid_s, mu_a, oit,
15                     fee, kappa, l_bar, psi, theta,
16                     delta_s, delta_b_size, N_q, q_max):
17     """
18     Evaluate all N_a ask prices for one LP type (v_val, r_val).
19     Defined at module level so joblib/loky can pickle it.
20     Returns (kv, kr, optimal_ask, mean_depth, best_V0).
21     """
22     def margin(a):
23         return a * (1 - fee) - v_val
24
25     u = rank_grid(N_q)
26     ni = 3 if oit < 10 else 5
27
28     best_V0 = -np.inf
29     best_j = 0
30     best_db = 0.0
31
32     for j in range(len(a_grid_s)):
33         if margin(a_grid_s[j]) <= 0.0005:
34             continue
35
36         q = np.linspace(0, q_max, N_q)
37         for _ in range(ni):
38             jp, jm = destination_indices(q, delta_s, delta_b_size)
39             V, l = solve_hjb(q, u, jp, jm,
40                             rho=r_val, kappa=kappa, l_bar=l_bar,
41                             psi=psi, theta=theta,
42                             margin=margin(a_grid_s[j]),
43                             db=delta_b_size,
44                             mu_exec=mu_a[j])
45             Psi = 1 + psi * u
46             Theta = 1 + theta * (1 - u)
47             m = stat_dist(jp, jm,
48                           l * Psi,
49                           mu_a[j] * np.ones_like(q) * Theta)
50             qt = quant_upd(u, q, m)
51             q = np.maximum(np.sort((1 - 0.15)*q + 0.15*qt), 0)

```

```

52
53     V0    = float(V[0])
54     dbar  = float(np.dot(m, q))
55
56     if V0 > best_V0:
57         best_V0 = V0
58         best_j  = j
59         best_db = dbar
60
61     return kv, kr, a_grid_s[best_j], best_db, best_V0

```

```

1  def solve_one_side(p_fund, alpha_beta, beta_beta, side_label):
2      """
3      Solve the Bertrand-MaNoH equilibrium for one side of the book (
4      YES or NO).
5      Returns grids and equilibrium objects for plotting.
6      """
7      # LP valuations live in [0, p_fund(1-fee)], drawn from Beta(
8      #   alpha, beta)
9      v_upper_bound = p_fund * (1 - fee)
10     # We want v in [0, v_upper_bound], concentrated by Beta density.
11     # Use inverse-CDF sampling from Beta restricted to [0,
12     #   v_upper_bound].
13     # Probability mass of Beta in [0, v_upper_bound]:
14     p_max_ask_side = beta_dist.cdf(v_upper_bound, alpha_beta,
15     #   beta_beta)
16     if p_max_ask_side < 0.05:
17         # Very few LPs on this side (e.g. extreme p); use uniform
18         #   fallback
19         v_grid_s = np.linspace(0.01, max(v_upper_bound - 0.005,
20         #   0.02), N_v)
21     else:
22         # Beta-quantile grid from 5th to 99th percentile of the
23         #   restricted Beta
24         probs = np.linspace(0.05, 0.99, N_v) * p_max_ask_side
25         v_grid_s = beta_dist.ppf(probs, alpha_beta, beta_beta)
26         v_grid_s = np.clip(v_grid_s, 0.005, v_upper_bound - 0.005)
27         v_grid_s = np.sort(np.unique(v_grid_s))
28         while len(v_grid_s) < N_v: # pad if unique dropped too many
29             v_grid_s = np.concatenate([v_grid_s, [v_grid_s[-1] -
30             #   0.001]])
31             v_grid_s = np.sort(np.unique(v_grid_s))
32
33     # Use geometric spacing (fine near p_fund, coarser far away).
34     a_lo = p_fund + 1e-4
35     a_hi = min(0.995, 1.0 - fee/2)
36     if a_lo >= a_hi:

```

```

29     a_lo = a_hi - 0.05
30     # Concentrate near p with a power-law spacing
31     t = np.linspace(0, 1, N_a) ** 1.5
32     a_grid_s = a_lo + (a_hi - a_lo) * t
33
34     def phi_s(a):
35         return lam_arr * (1.0 - beta_dist.cdf(a, alpha_beta,
36                                     beta_beta))
37
38     def margin_s(a, v):
39         return a * (1 - fee) - v
40
41     def solve_at(a_p, v_lp, rho, mu, ni=5):
42         mg = margin_s(a_p, v_lp)
43         if mg <= 0.0005:
44             return 0.0, 0.0
45         u = rank_grid(N_q)
46         q = np.linspace(0, q_max, N_q)
47         for _ in range(ni):
48             jp, jm = destination_indices(q, delta_s, delta_b_size)
49             V, l = solve_hjb(q, u, jp, jm, rho=rho, kappa=kappa,
50                             l_bar=l_bar,
51                             psi=psi, theta=theta, margin=mg, db=
52                             delta_b_size,
53                             mu_exec=mu)
54             Psi = 1 + psi*u; Theta = 1 + theta*(1 - u)
55             m = stat_dist(jp, jm, l*Psi, mu*np.ones_like(q)*Theta)
56             qt = quant_upd(u, q, m)
57             q = np.maximum(np.sort((1-0.15)*q + 0.15*qt), 0)
58         return float(V[0]), float(np.dot(m, q))
59
60     print(f"\n--- {side_label} side (p={p_fund:.3f}, Beta({
61         alpha_beta},{beta_beta})) ---")
62     print(f"    Ask grid: [{a_grid_s[0]:.4f}, {a_grid_s[-1]:.4f}] (N={
63         N_a})")
64     print(f"    LP v_grid: [{v_grid_s[0]:.3f}, {v_grid_s[-1]:.3f}] (N
65         ={N_v})")
66     print(f"    r_grid: [{r_grid[0]:.3f}, {r_grid[-1]:.3f}] (N={N_r})"
67         )
68
69     # --- Trapezoidal weights for non-uniform v_grid ---
70     dv_weights = np.zeros(N_v)
71     if N_v > 1:
72         dv_weights[0] = (v_grid_s[1] - v_grid_s[0]) / 2
73         dv_weights[-1] = (v_grid_s[-1] - v_grid_s[-2]) / 2
74         for k in range(1, N_v - 1):

```

```

68         dv_weights[k] = (v_grid_s[k+1] - v_grid_s[k-1]) / 2
69     else:
70         dv_weights[0] = 1.0
71
72     dr_weights = np.zeros(N_r)
73     if N_r > 1:
74         dr_weights[0] = (r_grid[1] - r_grid[0]) / 2
75         dr_weights[-1] = (r_grid[-1] - r_grid[-2]) / 2
76         for l in range(1, N_r - 1):
77             dr_weights[l] = (r_grid[l+1] - r_grid[l-1]) / 2
78     else:
79         dr_weights[0] = 1.0
80
81     # --- Outer Bertrand fixed point ---
82     D_a = np.ones(N_a) * 2.0
83     D_ces = np.zeros(N_a); ces_n = 0
84     a_star_2d = np.zeros((N_v, N_r))
85     d_bar_2d = np.zeros((N_v, N_r))
86     V0_best_2d = np.zeros((N_v, N_r))
87     t0 = time.time()
88
89     for oit in range(outer_max):
90         mu_a = phi_s(a_grid_s) / np.maximum(D_a, 0.01)
91
92         tasks = [
93             (kv, kr,
94              v_grid_s[kv], r_grid[kr],
95              a_grid_s, mu_a, oit,
96              fee, kappa, l_bar, psi, theta,
97              delta_s, delta_b_size, N_q, q_max)
98             for kv in range(N_v)
99             for kr in range(N_r)
100         ]
101         results = Parallel(n_jobs=-1, backend='loky')(
102             delayed(_solve_type_pair)(*t) for t in tasks
103         )
104         for kv, kr, a_star, d_bar, V0 in results:
105             a_star_2d[kv, kr] = a_star
106             d_bar_2d[kv, kr] = d_bar
107             V0_best_2d[kv, kr] = V0
108
109         # Aggregate D(a)
110         D_new = np.zeros(N_a)
111         step = np.diff(a_grid_s)
112         step_at = np.concatenate([[step[0]], (step[:-1] + step[1:])

```

```

113     for kv in range(N_v):
114         for kr in range(N_r):
115             fv = beta_dist.pdf(v_grid_s[kv], alpha_beta,
116                                beta_beta)
117             gr = 1.0 / (r_max - r_min)
118             w = N_total * fv * dv_weights[kv] * gr * dr_weights[
119                 kr] * d_bar_2d[kv, kr]
120             kern = np.exp(-0.5 * ((a_grid_s - a_star_2d[kv, kr])
121                                   / sigma_smooth)**2)
122             kern_norm = np.sum(kern * step_at) + 1e-20
123             D_new += w * kern / kern_norm
124
125     D_new = np.maximum(D_new, 0.005)
126     D_a = (1 - omega) * D_a + omega * D_new
127
128     if oit >= cesaro_start:
129         ces_n += 1
130         D_ces += (D_a - D_ces) / ces_n
131
132     if (oit + 1) % 5 == 0 or oit == 0:
133         mid = N_v // 2
134         print(f"   iter {oit+1:3d}: a*(v={v_grid_s[mid]:.2f}) in
135             "
136             f"[{a_star_2d[mid,:].min():.4f}, {a_star_2d[mid
137                 ,:].max():.4f}], "
138             f"D_max={D_a.max():.2f} [{time.time()-t0:.0f}s]")
139
140     D_final = D_ces if ces_n > 0 else D_a
141
142     # Best ask: lowest a_j where D_final > small threshold
143     thresh = 0.05 * D_final.max()
144     above_thresh = np.where(D_final > thresh)[0]
145     best_ask_idx = above_thresh[0] if len(above_thresh) > 0 else 0
146     best_ask = a_grid_s[best_ask_idx]
147
148     # Cumulative ask depth: integrate from the BEST ASK out to each
149     # price a_i.
150     # D_cum(a_i) = int_{a_best}^{a_i} d(s) ds for a_i >= a_best,
151     # else 0.
152     D_cum = np.zeros(N_a)
153     for i in range(N_a):
154         if i > best_ask_idx:
155             D_cum[i] = np.trapz(D_final[best_ask_idx:i+1], a_grid_s[
156                 best_ask_idx:i+1])
157         else:
158             D_cum[i] = 0.0

```

```

151
152     # Check: is best ask at the grid floor?
153     floor_binds = best_ask_idx <= 1
154     print(f"   Done [{time.time()-t0:.0f}s]. Best ask = {best_ask:.4f
155           }, "
156           f"floor binds: {floor_binds}")
157     print(f"   a*_min = {a_star_2d.min():.4f}, a*_max = {a_star_2d.
158           max():.4f}")
159
160     return (a_grid_s, D_final, D_cum, a_star_2d, d_bar_2d, v_grid_s,
161            best_ask, floor_binds, V0_best_2d)

```

```

1     print("="*60)
2     print("Full LOB: general asset   (ASK + BID sides)")
3     print("="*60)
4     print(f"p (norm) = {p_mid}   ->   real mid = {denorm(p_mid):.4f}")
5     print(f"support   = [{P_lo}, {P_hi}], fee = {fee}, valuation Beta({
6           alpha_F},{beta_F})")
7
8     # ASK side: sell-limit LPs with valuation below the mid; margin = a
9       (1-f) - v.
10
11     (a_yes, d_yes, dc_yes, as_yes, db_yes, v_yes,
12      ba_yes, floor_yes, V0_yes) = solve_one_side(p_mid, alpha_F, beta_F,
13      'ASK')
14
15     # BID side: buy-limit LPs with valuation ABOVE the mid; margin = v -
16       b.
17
18     (a_no, d_no, dc_no, as_no, db_no, v_no,
19      ba_no, floor_no, V0_no) = solve_one_side(1 - p_mid, beta_F, alpha_F
20      , 'BID')
21
22     # Reflect the BID side (reflected ask) book back into bid prices on
23       the asset.
24
25     b_yes_grid = 1.0 - a_no[::-1]
26     d_bid = d_no[::-1]
27     b_best_yes = 1.0 - ba_no                                # best bid (closest
28       to mid)
29     best_bid_idx = int(np.argmin(np.abs(b_yes_grid - b_best_yes)))
30
31     # Cumulative bid depth: integrate from price b_i UP TO the best bid.
32     dc_bid = np.zeros(len(b_yes_grid))
33     for i in range(len(b_yes_grid)):
34         if i < best_bid_idx:
35             dc_bid[i] = np.trapz(d_bid[i:best_bid_idx+1], b_yes_grid[i:
36               best_bid_idx+1])
37         else:
38             dc_bid[i] = 0.0

```

```

28
29 spread      = ba_yes + ba_no - 1.0
30 best_ask - best_bid (normalised)
31 spread_real = spread * (P_hi - P_lo)           # spread in real
    price units
32
33 print(f"\n{'='*60}")
34 print(f"EQUILIBRIUM RESULTS")
35 print(f"{'='*60}")
36 print(f"Best ask  = {ba_yes:.4f} (norm) = {denorm(ba_yes):.4f} (real
    )  "
37       f"[floor binds: {floor_yes}]")
38 print(f"Best bid  = {b_best_yes:.4f} (norm) = {denorm(b_best_yes):.4
    f} (real)  "
39       f"[floor binds: {floor_no}]")
40 print(f"Mid      = {p_mid:.4f} (norm) = {denorm(p_mid):.4f} (real)"
    )
41 print(f"Spread    = {spread:.4f} (norm) = {spread_real:.4f} (real)")

```


References

- Avellaneda, M. and Stoikov, S. (2008). High-frequency trading in a limit order book. *Quantitative Finance*, 8(3):217–224.
- Chan, P. and Sircar, R. (2015). Bertrand and Cournot mean field games. *Applied Mathematics & Optimization*, 71(3):533–569.
- Cont, R., Stoikov, S., and Talreja, R. (2010). A stochastic model for order book dynamics. *Operations Research*, 58(3):549–563.
- Glosten, L. R. and Milgrom, P. R. (1985). Bid, ask and transaction prices in a specialist market with heterogeneously informed traders. *Journal of Financial Economics*, 14(1):71–100.
- Gromb, D. and Vayanos, D. (2010). Limits of arbitrage. *Annual Review of Financial Economics*, 2:251–275.
- Guéant, O., Lehalle, C.-A., and Fernandez-Tapia, J. (2013). Dealing with the inventory risk: A solution to the market making problem. *Mathematics and Financial Economics*, 7(4):477–507.
- Huang, M., Caines, P. E., and Malhamé, R. P. (2006). Large population stochastic dynamic games: Closed-loop McKean–Vlasov systems and the Nash certainty equivalence principle. *Communications in Information & Systems*, 6(3):221–252.
- Lasry, J.-M. and Lions, P.-L. (2007). Mean field games. *Japanese Journal of Mathematics*, 2(1):229–260.
- Ma, J. and Noh, E. (2020). Equilibrium model of limit order books – a mean-field game view. *arXiv preprint*, arXiv:2002.12857. arXiv:2002.12857 [math.OC].
- Ma, J., Wang, X., and Zhang, J. (2015). Dynamic equilibrium limit order book model and optimal execution problem. *Mathematical Control and Related Fields*, 5(3).
- Shleifer, A. and Vishny, R. W. (1997). The limits of arbitrage. *Journal of Finance*, 52(1):35–55.
- Touzo, L., Marsili, M., and Zagier, D. (2021). Information thermodynamics of financial markets: The Glosten–Milgrom model. *arXiv preprint*. arXiv:2010.01905 [cond-mat.stat-mech].
- Zeng, S., Evans, B. P., Bhatt, S., Ardon, L., Ganesh, S., and Koppel, A. (2025). Learning in Stackelberg mean field games: A non-asymptotic analysis. In *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv:2509.15392.